



Compiladores

Aula 8: Análise Sintática Ascendente

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Shift-reduce
- Itens LR
- Tabelas ACTION/GOTO

Shift-reduce

- Uma GLC não apenas orienta a expansão, mas agora o parser reconstrói a derivação a partir da entrada.
- Na análise descendente, a árvore era construída de cima para baixo. Aqui, os fragmentos reconhecidos são combinados até formar o símbolo inicial.

Shift-reduce

- Etapas da análise ascendente.

Partida

Tokens produzidos pelo scanner e uma pilha que registra o contexto do reconhecimento.

Transformação

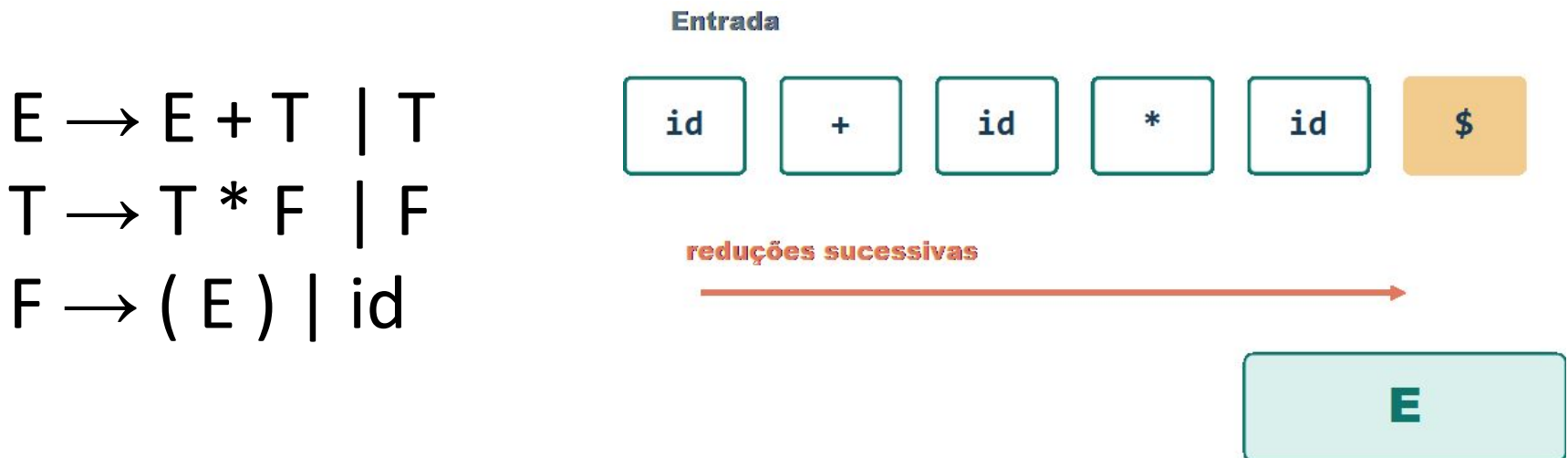
Deslocar símbolos, reduzir fragmentos e consultar estados para decidir o próximo passo.

Chegada

Reconhecer a sentença, construir a AST e relatar erros sintáticos localizados.

Shift-reduce

- A análise começa pelos tokens e combina fragmentos reconhecidos até formar estruturas maiores.



A sequência de reduções corresponde à derivação mais à direita lida de trás para frente.

Shift-reduce

- As quatro ações do parser.

Ação	Significado	Exemplo
shift s5	Desloca o token e vai ao estado 5.	Ler id e empilhá-lo.
reduce $A \rightarrow \beta$	Retira β , empilha A e consulta GOTO.	Reduz id para F.
accept	A entrada foi reconhecida integralmente.	Chegou a \$ com a produção inicial.
error	Não há ação válida para estado e token.	Token inesperado.

Reduzir não apaga informação: transforma uma sequência em um não-terminal, preservando a estrutura necessária para continuar.

Shift-reduce

- Exemplo:

Gramática reduzida: $E' \rightarrow E$; $E \rightarrow E + T \mid T$; $T \rightarrow id$.

Pilha	Entrada	Ação
$\emptyset$	id + id \$	shift 2
$\emptyset \text{ id } 2$	+ id \$	reduce $T \rightarrow id$
$\emptyset \text{ T } 1$	+ id \$	reduce $E \rightarrow T$
$\emptyset \text{ E } 3$	+ id \$	shift 4
$\emptyset \text{ E } 3 + 4$	id \$	shift 2
$\emptyset \text{ E } 3 + 4 \text{ id } 2$	\$	reduce $T \rightarrow id$
$\emptyset \text{ E } 3 + 4 \text{ T } 5$	\$	reduce $E \rightarrow E + T$
$\emptyset \text{ E } 3$	\$	accept

Observe: primeiro os operandos, depois T, em seguida E e, por fim, a aceitação.

Itens LR

- Um item (LR) registra quanto de uma produção já foi reconhecido.

$E \rightarrow E \bullet + T$

$E \rightarrow E + \bullet T$

$E \rightarrow E + T \bullet$



fechamento

Se o ponto está antes de um não-terminal, suas produções entram no conjunto de itens.

GOTO

Desloca o ponto sobre um símbolo e calcula o próximo estado.

O ponto avança quando o parser desloca um símbolo. No fim da produção, sugere uma redução.

Tabelas ACTION/GOTO

- ACTION decide sobre terminais;
- GOTO conduz as reduções entre estados.

Estado	id	+	\$	E	T
0	s2	–	–	1	3
1	–	s4	acc	–	–
2	–	r3	r3	–	–
3	–	r2	r2	–	–
4	s2	–	–	–	5
5	–	r1	r1	–	–

sN / rN

shift para estado N; reduzir pela produção N.

Conflitos

shift/reduce ou reduce/reduce exigem diagnóstico.

—
célula não definida: erro.

Tabelas ACTION/GOTO

- SLR(1), LR(1) e LALR(1)
- Quanto mais contexto o item carrega, maior o poder de decisão — e maior a complexidade.

Técnica	Contexto usado	Papel didático
LR(0)	Itens sem lookahead.	Mecanismo básico.
SLR(1)	FOLLOW restringe reduções.	Ponte entre teoria e implementação.
LR(1)	Lookaheads específicos por item.	Maior poder de reconhecimento.
LALR(1)	Agrupar estados LR(1) compatíveis.	Muito comum em geradores.

Não esconda conflitos: registre estado, token, ações candidatas e decisão adotada.

Tabelas ACTION/GOTO

- O parser ascendente transforma decisões locais em uma estrutura global.
- A pilha registra o contexto;
- ACTION escolhe shift, reduce, accept ou error;
- GOTO conduz a continuação após cada redução.



Aplicação no projeto MINIC

- Os tokens da Etapa 1 entram → as reduções da Etapa 2 constroem a AST.

```

programa → função
função → INT MAIN ( ) bloco
bloco → { lista_comandos }
comando → declaração ;
          | atribuição ;
          | RETURN expressão ;
    
```

fator → NUM

→ **Literal(valor)**

fator → ID

→ **Identifier(nome)**

expressão → expressão + termo

→ **Binary(PLUS, esq., dir.)**

comando → ID = expressão ;

→ **Assignment(nome, expr.)**

Cada redução pode disparar a criação de um nó da AST.

Aplicação no projeto MINIC

- Tratamento de erros sintáticos.
- A mensagem deve localizar o problema e ajudar a recuperar o reconhecimento.

`int x = 2 + ;`
 linha 1, coluna 13:
 expressão esperada após "+";
 encontrado ";".

Pânico

Descartar tokens até ";" ou "}".

Local

Inserir/remover token quando inequívoco.

Erro

Produções específicas para orientar a recuperação.

- estado atual e token de lookahead
- pilha e célula ACTION consultada
- produção escolhida e posição no código-fonte

Aplicação no projeto MINIC

- Casos de aceitação e rejeição.
- Testar o mecanismo significa verificar tanto o reconhecimento quanto a explicação do erro.

Categoria	Exemplo
Válido	<code>int x = 2 + 3;</code>
Precedência	<code>x = 2 + 3 * 4;</code>
Parênteses	<code>x = (2 + 3) * 4;</code>
Token ausente	<code>x = 2 + ;</code>
Delimitador ausente	<code>int x = 2</code>
Uso do parser	<code>print(x);</code>

Reconhecer

A entrada válida deve produzir a AST esperada.

Rejeitar

A entrada inválida deve indicar posição e expectativa.

Recuperar

Evitar loops e mascaramento da causa original.